READ DATA

```
import os
import random
from google.colab import drive

drive.mount('/content/drive')
train_pneumonia_dir = '/content/drive/MyDrive/TA/Dataset/train/PNEUMONIA'
test_pneumonia_dir = '/content/drive/MyDrive/TA/Dataset/test/PNEUMONIA'
train_normal_dir = '/content/drive/MyDrive/TA/Dataset/train/NORMAL'
test_normal_dir = '/content/drive/MyDrive/TA/Dataset/test/NORMAL'
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
def get_all_files(dirs, allowed_extensions=['.jpg', '.jpeg', '.png']):
    all_files = []
    for dir_path in dirs:
        files = [os.path.join(dir_path, f) for f in os.listdir(dir_path)
                 if not f.startswith('.') and os.path.splitext(f)[1].lower() in allowed_extensions]
        all_files.extend(files)
    return all_files

def random_select_files(file_list, subset_size):
    if subset_size > len(file_list):
        raise ValueError("Jumlah file yang diminta lebih besar daripada jumlah file yang tersedia.")
    return random.sample(file_list, subset_size)

def count_files(directory):
  return len(os.listdir(directory))
```

```
all_pneumonia_files = get_all_files([train_pneumonia_dir, test_pneumonia_dir])
all_normal_files = get_all_files([train_normal_dir, test_normal_dir])

print(f"train pneumon : {count_files(train_pneumonia_dir)}")
print(f"test pneumon : {count_files(test_pneumonia_dir)}")
print(f"train norm : {count_files(train_normal_dir)}")
print(f"test norm : {count_files(test_normal_dir)}")

print(f"pneumon : {len(all_pneumonia_files)}")
```

```
print(f"norm : {len(all_normal_files)}")
```

```
train pneumon : 3875
test pneumon : 390
train norm : 1341
test norm : 234
pneumon : 4265
norm : 1575
```

```
train_pneumonia_samples = random_select_files(all_pneumonia_files, 1181)
test_pneumonia_samples = random_select_files([f for f in all_pneumonia_files if f not in train_pneumonia_samples], 394)

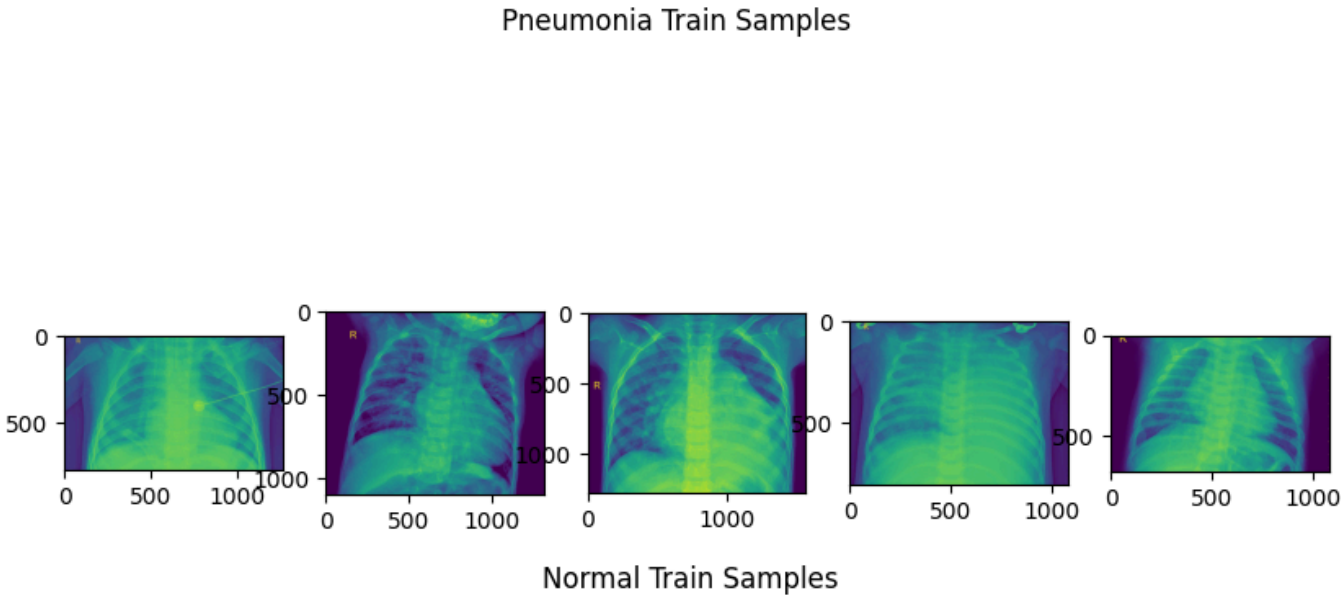
train_normal_samples = random_select_files(all_normal_files, 1181)
test_normal_samples = random_select_files([f for f in all_normal_files if f not in train_normal_samples], 394)

#train_pneumonia_samples = [os.path.join(train_pneumonia_dir, file) for file in train_pneumonia_files]
#test_pneumonia_samples = [os.path.join(train_pneumonia_dir, file) for file in test_pneumonia_files]
#train_normal_samples = [os.path.join(train_normal_dir, file) for file in train_normal_files]
#test_normal_samples = [os.path.join(train_normal_dir, file) for file in test_normal_files]
```

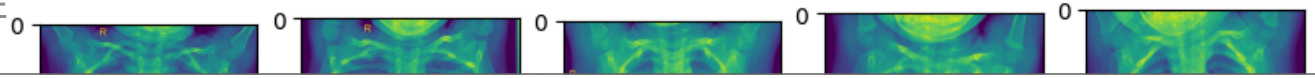
```
from PIL import Image
import matplotlib.pyplot as plt

def show_sample_images(sample_files,ammount, title):
    plt.figure(figsize=(10, 5))
    plt.suptitle(title)
    for i in range(ammount):
        plt.subplot(1, ammount, i+1)
        img = Image.open(sample_files[i])
        plt.imshow(img)
    plt.show()

show_sample_images(train_pneumonia_samples, 5, 'Pneumonia Train Samples')
show_sample_images(train_normal_samples, 5, 'Normal Train Samples')
show_sample_images(test_pneumonia_samples, 5, 'Pneumonia Test Samples')
show_sample_images(test_normal_samples, 5, 'Normal Test Samples')
```



RESIZE

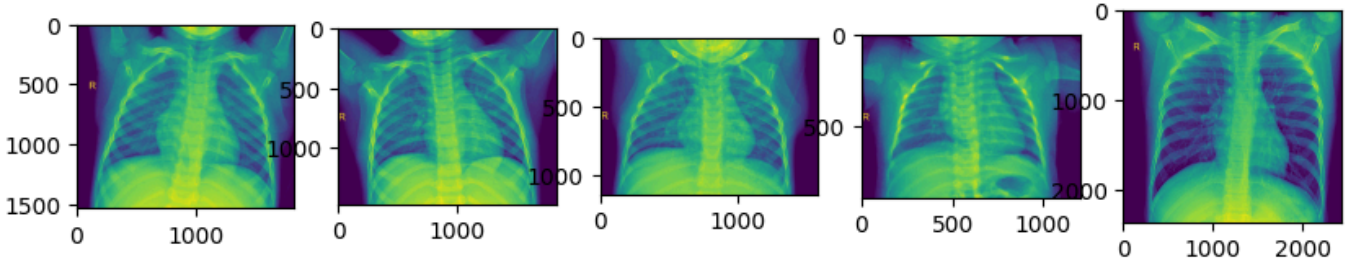


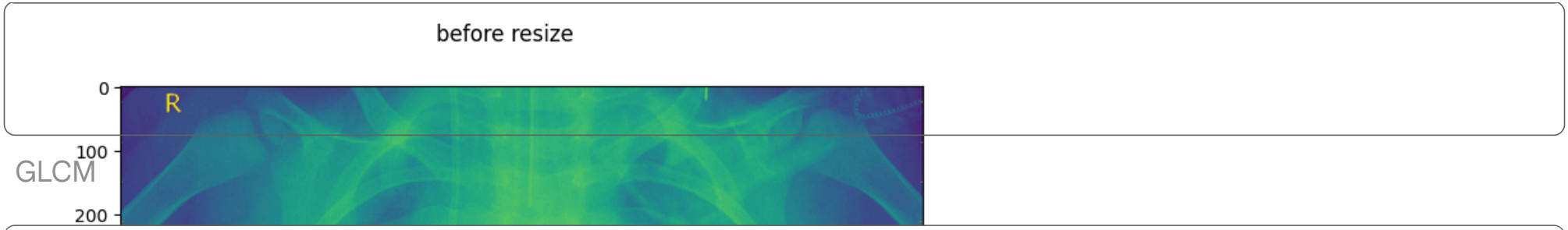
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
import numpy as np
from tensorflow.keras.preprocessing.image import load_img, img_to_array

def load_images(file_paths, target_size=(224, 224)):
    images = []
    for i, file_path in enumerate(file_paths):
        img = load_img(file_path, target_size=target_size)
        img_array = img_to_array(img)
        img_array = img_array - 30
        img_array = img_array / 255.0
        images.append(img_array)
    return images
```

```
train_pneumonia_images = load_images(train_pneumonia_samples)
test_pneumonia_images = load_images(test_pneumonia_samples)
train_normal_images = load_images(train_normal_samples)
test_normal_images = load_images(test_normal_samples)
```

```
show_sample_images(train_pneumonia_samples, 1, 'before resize')
plt.imshow(train_pneumonia_images[0])
plt.title('After Resizing')
```





GLCM

```
import numpy as np
import cv2
from skimage.feature import graycomatrix, graycoprops

def calculate_glcm(image):
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    distances = [1, 2, 3]
    angles = [0, np.pi/4, np.pi/2, 3*np.pi/4]

    glcm = graycomatrix(gray_image,
                        distances=distances,
                        angles=angles,
                        levels=256,
                        symmetric=True,
                        normed=True)

    properties = ['contrast', 'dissimilarity', 'homogeneity', 'energy', 'correlation', 'ASM']
    features = {}

    for prop in properties:
        values = graycoprops(glcm, prop).flatten()
        features[f'{prop}_mean'] = np.mean(values)
        features[f'{prop}_std'] = np.std(values)
        features[f'{prop}_var'] = np.var(values)

    glcm_entropy = -np.sum(glcm * np.log2(glcm + 1e-10))
    features['entropy'] = glcm_entropy

    return features
```

```
import cv2

def extract_glcm_features(image_list):
    glcm_features_list = []
```

```
    for image in image_list:
        image_uint8 = (image * 255).astype(np.uint8)
        features = calculate_glcm(image_uint8)
        glcm_features_list.append(features)
    return glcm_features_list

train_pneumonia_glcm = extract_glcm_features(train_pneumonia_images)
test_pneumonia_glcm = extract_glcm_features(test_pneumonia_images)
train_normal_glcm = extract_glcm_features(train_normal_images)
test_normal_glcm = extract_glcm_features(test_normal_images)

print("GLCM features for the first pneumonia training image:")
train_pneumonia_glcm[0]
```

```
GLCM features for the first pneumonia training image:
{'contrast_mean': np.float64(879.4891920502932),
 'contrast_std': np.float64(409.8571537549472),
 'contrast_var': np.float64(167982.88648410642),
 'dissimilarity_mean': np.float64(9.440646671475902),
 'dissimilarity_std': np.float64(3.039878415614898),
 'dissimilarity_var': np.float64(9.240860781721343),
 'homogeneity_mean': np.float64(0.21511625437692347),
 'homogeneity_std': np.float64(0.04415300969369455),
 'homogeneity_var': np.float64(0.0019494882650114853),
 'energy_mean': np.float64(0.02056754494921029),
 'energy_std': np.float64(0.0022670317989337),
 'energy_var': np.float64(5.139433177376568e-06),
 'correlation_mean': np.float64(0.8319009182837901),
 'correlation_std': np.float64(0.0795653176843357),
 'correlation_var': np.float64(0.006330639778209263),
 'ASM_mean': np.float64(0.00042816333841516233),
 'ASM_std': np.float64(9.555068625468819e-05),
 'ASM_var': np.float64(9.129933643741858e-09),
 'entropy': np.float64(144.30052440984124)}
```

```
#import pandas as pd
#from google.colab import files

#train_pneumonia_df = pd.DataFrame(train_pneumonia_glcm)
#train_pneumonia_df['label'] = 'PNEUMONIA'

#test_pneumonia_df = pd.DataFrame(test_pneumonia_glcm)
#test_pneumonia_df['label'] = 'PNEUMONIA'

#train_normal_df = pd.DataFrame(train_normal_glcm)
#train_normal_df['label'] = 'NORMAL'
```

```
#test_normal_df = pd.DataFrame(test_normal_glcm)
#test_normal_df['label'] = 'NORMAL'

#all_glcm_df = pd.concat([train_pneumonia_df, test_pneumonia_df, train_normal_df, test_normal_df], ignore_index=True)

#excel_filename = 'glcm_features.xlsx'
#all_glcm_df.to_excel(excel_filename, index=False)

#files.download(excel_filename)

#print(f"GLCM features saved to '{excel_filename}' and downloaded.")
```

CNN

```
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.regularizers import l2

# GLCM - DF
train_pneumonia_glcm_df = pd.DataFrame(train_pneumonia_glcm)
test_pneumonia_glcm_df = pd.DataFrame(test_pneumonia_glcm)
train_normal_glcm_df = pd.DataFrame(train_normal_glcm)
test_normal_glcm_df = pd.DataFrame(test_normal_glcm)

train_pneumonia_glcm_df['label'] = 1  # 1 for pneumonia
test_pneumonia_glcm_df['label'] = 1
train_normal_glcm_df['label'] = 0  # 0 for normal
test_normal_glcm_df['label'] = 0

train_glcm_df = pd.concat([train_pneumonia_glcm_df, train_normal_glcm_df], ignore_index=True)
test_glcm_df = pd.concat([test_pneumonia_glcm_df, test_normal_glcm_df], ignore_index=True)


# Separate features
X_train = train_glcm_df.drop('label', axis=1)
y_train = train_glcm_df['label']
X_test = test_glcm_df.drop('label', axis=1)
y_test = test_glcm_df['label']
```

```
# Normal
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Model
model = Sequential([
    Dense(128, activation='relu', kernel_regularizer=l2(0.0001), input_shape=(X_train.shape[1],)),
    BatchNormalization(),
    Dropout(0.3),
    Dense(64, activation='relu', kernel_regularizer=l2(0.0001)),
    BatchNormalization(),
    Dropout(0.3),
    Dense(32, activation='relu', kernel_regularizer=l2(0.0001)),
    BatchNormalization(),
    Dropout(0.3),
    Dense(1, activation='sigmoid')
])

optimizer = Adam(learning_rate=0.001)
class_weights = {0: 1.5, 1: 0.8}

callbacks = [
    EarlyStopping(monitor='val_loss', patience=50, restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.3, patience=25, min_lr=1e-6)
]

model.compile(optimizer=optimizer,
              loss='binary_crossentropy',
              metrics=['accuracy', tf.keras.metrics.AUC()])

model.summary()

history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.3,
    callbacks=None,
    class_weight=class_weights,
)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 128)	2,560
batch_normalization_3 (BatchNormalization)	(None, 128)	512
dropout_3 (Dropout)	(None, 128)	0
dense_5 (Dense)	(None, 64)	8,256
batch_normalization_4 (BatchNormalization)	(None, 64)	256
dropout_4 (Dropout)	(None, 64)	0
dense_6 (Dense)	(None, 32)	2,080
batch_normalization_5 (BatchNormalization)	(None, 32)	128
dropout_5 (Dropout)	(None, 32)	0
dense_7 (Dense)	(None, 1)	33

```
 Total params: 13,825 (54.00 KB)
 Trainable params: 13,377 (52.25 KB)
 Non-trainable params: 448 (1.75 KB)
Epoch 1/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 4s 16ms/step - accuracy: 0.6483 - auc_1: 0.7176 - loss: 0.6857 - val_accuracy: 0.6403 - val_auc_1: 0.0000e+00 - val_loss: 0.689
Epoch 2/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8071 - auc_1: 0.8995 - loss: 0.4392 - val_accuracy: 0.6164 - val_auc_1: 0.0000e+00 - val_loss: 0.707
Epoch 3/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8409 - auc_1: 0.9193 - loss: 0.3924 - val_accuracy: 0.7066 - val_auc_1: 0.0000e+00 - val_loss: 0.638
Epoch 4/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.8675 - auc_1: 0.9206 - loss: 0.3926 - val_accuracy: 0.7715 - val_auc_1: 0.0000e+00 - val_loss: 0.584
Epoch 5/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.8504 - auc_1: 0.9262 - loss: 0.3702 - val_accuracy: 0.8166 - val_auc_1: 0.0000e+00 - val_loss: 0.517
Epoch 6/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8606 - auc_1: 0.9212 - loss: 0.3836 - val_accuracy: 0.8533 - val_auc_1: 0.0000e+00 - val_loss: 0.433
Epoch 7/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8718 - auc_1: 0.9426 - loss: 0.3290 - val_accuracy: 0.8590 - val_auc_1: 0.0000e+00 - val_loss: 0.391
Epoch 8/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8740 - auc_1: 0.9433 - loss: 0.3250 - val_accuracy: 0.8886 - val_auc_1: 0.0000e+00 - val_loss: 0.342
Epoch 9/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8810 - auc_1: 0.9340 - loss: 0.3469 - val_accuracy: 0.8900 - val_auc_1: 0.0000e+00 - val_loss: 0.345
Epoch 10/50
```

```
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8787 - auc_1: 0.9359 - loss: 0.3417 - val_accuracy: 0.8970 - val_auc_1: 0.0000e+00 - val_loss: 0.3538
Epoch 11/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8902 - auc_1: 0.9445 - loss: 0.3165 - val_accuracy: 0.9055 - val_auc_1: 0.0000e+00 - val_loss: 0.3409
Epoch 12/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8868 - auc_1: 0.9463 - loss: 0.3081 - val_accuracy: 0.9069 - val_auc_1: 0.0000e+00 - val_loss: 0.3339
Epoch 13/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8896 - auc_1: 0.9484 - loss: 0.3049 - val_accuracy: 0.8970 - val_auc_1: 0.0000e+00 - val_loss: 0.3367
Epoch 14/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8901 - auc_1: 0.9430 - loss: 0.3155 - val_accuracy: 0.8928 - val_auc_1: 0.0000e+00 - val_loss: 0.3516
Epoch 15/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8709 - auc_1: 0.9378 - loss: 0.3363 - val_accuracy: 0.8999 - val_auc_1: 0.0000e+00 - val_loss: 0.3290
Epoch 16/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8937 - auc_1: 0.9516 - loss: 0.2915 - val_accuracy: 0.8984 - val_auc_1: 0.0000e+00 - val_loss: 0.3413
Epoch 17/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8918 - auc_1: 0.9471 - loss: 0.3075 - val_accuracy: 0.9168 - val_auc_1: 0.0000e+00 - val_loss: 0.3164
Epoch 18/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8922 - auc_1: 0.9452 - loss: 0.3073 - val_accuracy: 0.9055 - val_auc_1: 0.0000e+00 - val_loss: 0.3399
Epoch 19/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8955 - auc_1: 0.9469 - loss: 0.2983 - val_accuracy: 0.9126 - val_auc_1: 0.0000e+00 - val_loss: 0.3309
Epoch 20/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8798 - auc_1: 0.9462 - loss: 0.3027 - val_accuracy: 0.9097 - val_auc_1: 0.0000e+00 - val_loss: 0.3265
Epoch 21/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8791 - auc_1: 0.9449 - loss: 0.3070 - val_accuracy: 0.9111 - val_auc_1: 0.0000e+00 - val_loss: 0.3289
Epoch 22/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8838 - auc_1: 0.9495 - loss: 0.2995 - val_accuracy: 0.9168 - val_auc_1: 0.0000e+00 - val_loss: 0.3199
Epoch 23/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8954 - auc_1: 0.9560 - loss: 0.2690 - val_accuracy: 0.9126 - val_auc_1: 0.0000e+00 - val_loss: 0.3191
Epoch 24/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8970 - auc_1: 0.9502 - loss: 0.2898 - val_accuracy: 0.9083 - val_auc_1: 0.0000e+00 - val_loss: 0.3416
Epoch 25/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8854 - auc_1: 0.9481 - loss: 0.2936 - val_accuracy: 0.9210 - val_auc_1: 0.0000e+00 - val_loss: 0.3037
Epoch 26/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8820 - auc_1: 0.9492 - loss: 0.2999 - val_accuracy: 0.9097 - val_auc_1: 0.0000e+00 - val_loss: 0.3342
Epoch 27/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9030 - auc_1: 0.9546 - loss: 0.2761 - val_accuracy: 0.9111 - val_auc_1: 0.0000e+00 - val_loss: 0.3230
Epoch 28/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8968 - auc_1: 0.9567 - loss: 0.2681 - val_accuracy: 0.9083 - val_auc_1: 0.0000e+00 - val_loss: 0.3329
Epoch 29/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8735 - auc_1: 0.9407 - loss: 0.3136 - val_accuracy: 0.9111 - val_auc_1: 0.0000e+00 - val_loss: 0.3161
Epoch 30/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8847 - auc_1: 0.9538 - loss: 0.2776 - val_accuracy: 0.9097 - val_auc_1: 0.0000e+00 - val_loss: 0.3203
Epoch 31/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9000 - auc_1: 0.9595 - loss: 0.2616 - val_accuracy: 0.9069 - val_auc_1: 0.0000e+00 - val_loss: 0.3416
Epoch 32/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9009 - auc_1: 0.9554 - loss: 0.2694 - val_accuracy: 0.9083 - val_auc_1: 0.0000e+00 - val_loss: 0.3201
Epoch 33/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8894 - auc_1: 0.9522 - loss: 0.2880 - val_accuracy: 0.9097 - val_auc_1: 0.0000e+00 - val_loss: 0.3171
Epoch 34/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8931 - auc_1: 0.9552 - loss: 0.2724 - val_accuracy: 0.9111 - val_auc_1: 0.0000e+00 - val_loss: 0.3196
Epoch 35/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8996 - auc_1: 0.9610 - loss: 0.2563 - val_accuracy: 0.9126 - val_auc_1: 0.0000e+00 - val_loss: 0.3211
```

```
from sklearn.metrics import precision_score, recall_score
y_pred = model.predict(X_test)
y_pred_classes = (y_pred > 0.5).astype(int)
test_loss, test_accuracy, test_auc = model.evaluate(X_test, y_test, verbose=0)
test_precision = precision_score(y_test, y_pred_classes)
test_recall = recall_score(y_test, y_pred_classes)

print(f"- Loss: {test_loss:.4f}")
print(f"- AUC: {test_auc:.4f}")
print(f"- Accuracy: {test_accuracy:.4f}")
```

```
Epoch 42/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8905 - auc_1: 0.9555 - loss: 0.2814 - val_accuracy: 0.8970 - val_auc_1: 0.0000e+00 - val_loss: 0.3591
- Loss: 0.2975
Epoch 43/50
- AUC: 0.9498
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.9097 - auc_1: 0.9650 - loss: 0.2449 - val_accuracy: 0.9126 - val_auc_1: 0.0000e+00 - val_loss: 0.3237
- Accuracy: 0.8972
Epoch 44/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.9074 - auc_1: 0.9610 - loss: 0.2569 - val_accuracy: 0.8984 - val_auc_1: 0.0000e+00 - val_loss: 0.3665
```

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_classes))
```

```
Epoch 47/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8981 - auc_1: 0.9606 - loss: 0.2559 - val_accuracy: 0.9069 - val_auc_1: 0.0000e+00 - val_loss: 0.3275
Epoch 48/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8991 - auc_1: 0.9589 - loss: 0.2691 - val_accuracy: 0.8956 - val_auc_1: 0.0000e+00 - val_loss: 0.3662
Epoch 49/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8870 - auc_1: 0.9510 - loss: 0.2857 - val_accuracy: 0.9069 - val_auc_1: 0.0000e+00 - val_loss: 0.3429
Epoch 50/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8943 - auc_1: 0.9578 - loss: 0.2692 - val_accuracy: 0.9013 - val_auc_1: 0.0000e+00 - val_loss: 0.3648
```

```
              precision    recall  f1-score   support

           0       0.89      0.90      0.90       394
           1       0.90      0.89      0.90       394

    accuracy                           0.90       788
   macro avg       0.90      0.90      0.90       788
weighted avg       0.90      0.90      0.90       788
```

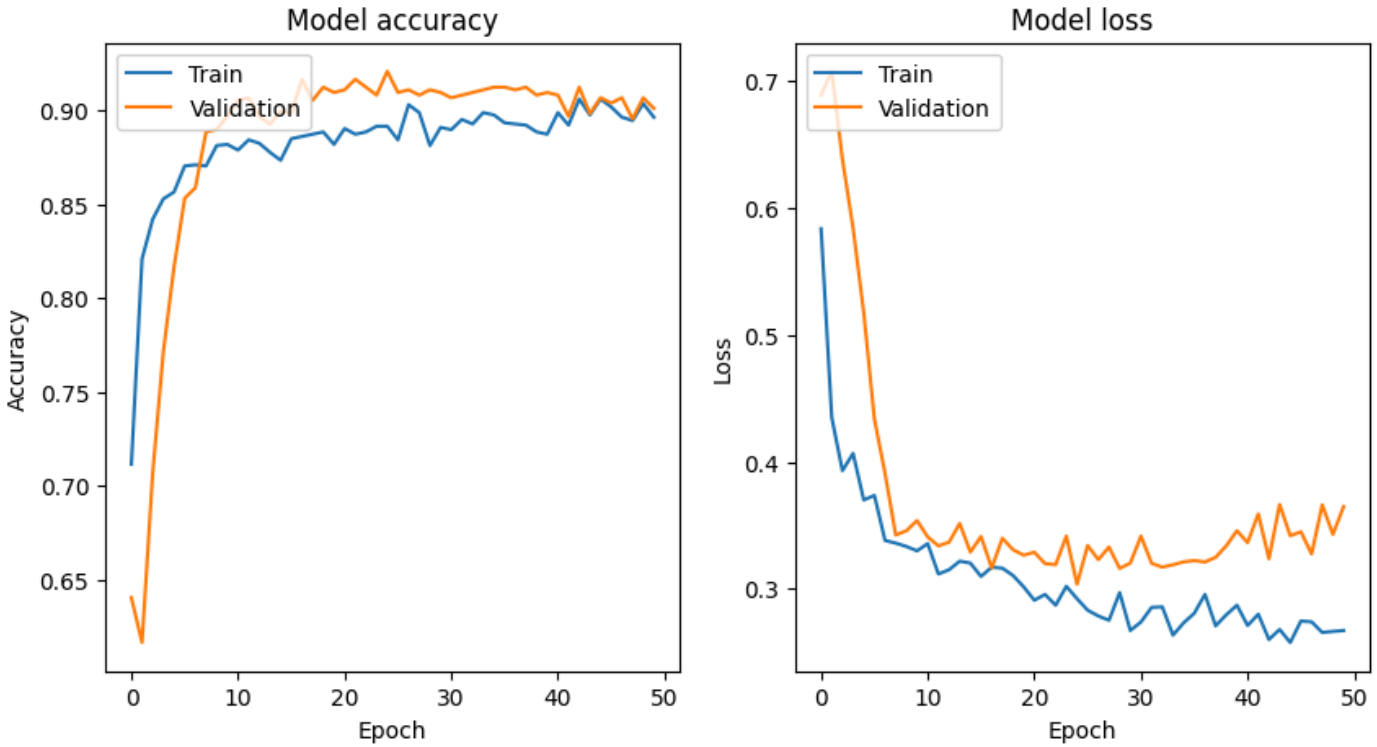
```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model loss')
```

```
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.show()
```



PREDICT

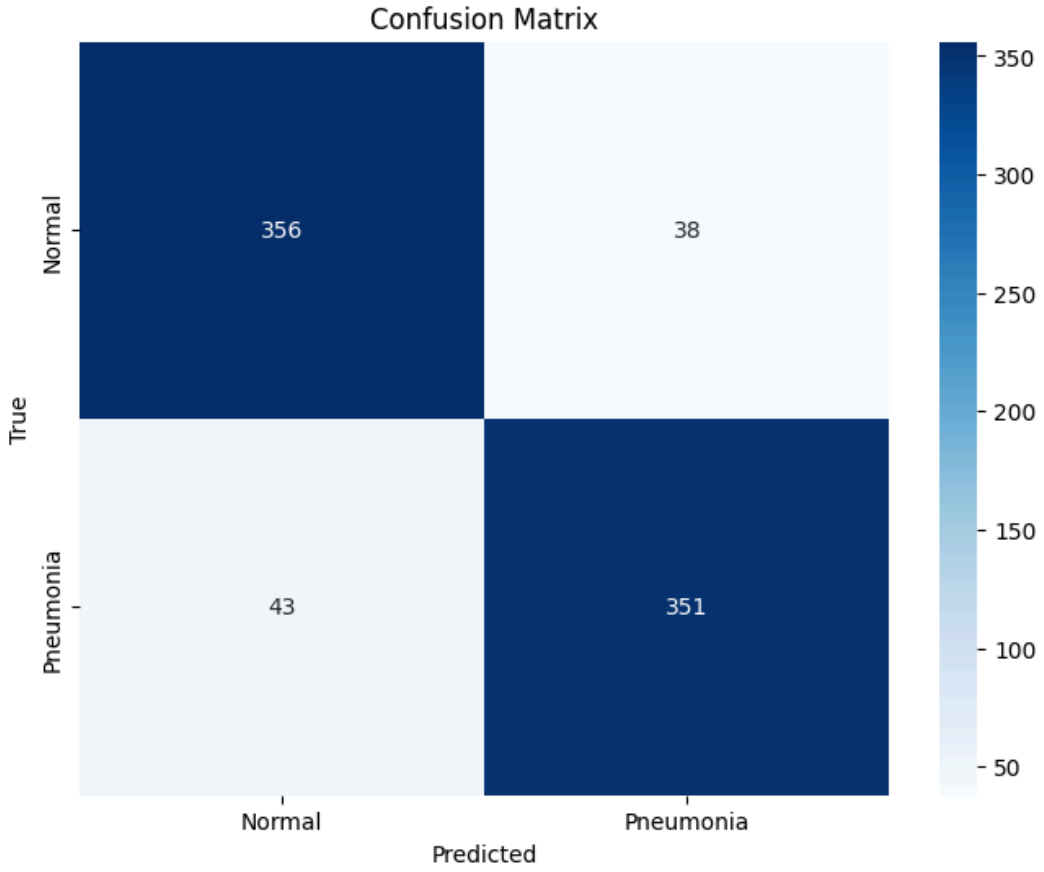
```
y_pred = model.predict(X_test)
y_pred_classes = (y_pred > 0.5).astype(int)
```

```
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
```

```
conf_matrix = confusion_matrix(y_test, y_pred_classes)

plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Pneumonia'], yticklabels=['Normal', 'Pneumonia'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()
```



CODE END HERE

```
import matplotlib.pyplot as plt
import cv2
import pandas as pd
import os
import random

test_dir = '/content/drive/MyDrive/TA/Dataset/val/NORMAL'
image_files = [os.path.join(test_dir, f) for f in os.listdir(test_dir) if os.path.isfile(os.path.join(test_dir, f))]

image_paths = []
predictions = []

for image_file in image_files:
    try:
        new_image = load_img(image_file, target_size=(150, 150))
        new_image_array = img_to_array(new_image)
        new_image_array = np.clip(new_image_array - 30, 0, 255)
        new_image_array = new_image_array / 255.0
        new_image_glcm = extract_glcm_features([new_image_array])
        new_image_glcm_df = pd.DataFrame(new_image_glcm)
        new_image_scaled = scaler.transform(new_image_glcm_df)
        prediction = model.predict(new_image_scaled)
        predicted_class = (prediction > 0.5).astype(int)[0][0]

        image_paths.append(image_file)
        predictions.append('Pneumonia' if predicted_class == 1 else 'Normal')

    except Exception as e:
        print(f"Error processing {image_file}: {e}")

results_df = pd.DataFrame({'Image Path': image_paths, 'Prediction': predictions})
print(results_df)

sample_files = random.sample(image_files, 5)

fig, axes = plt.subplots(1, 5, figsize=(20, 5))
fig.suptitle('Sample Predictions', fontsize=16)

for i, image_file in enumerate(sample_files):
    try:
        img = cv2.imread(image_file, cv2.IMREAD_GRAYSCALE)

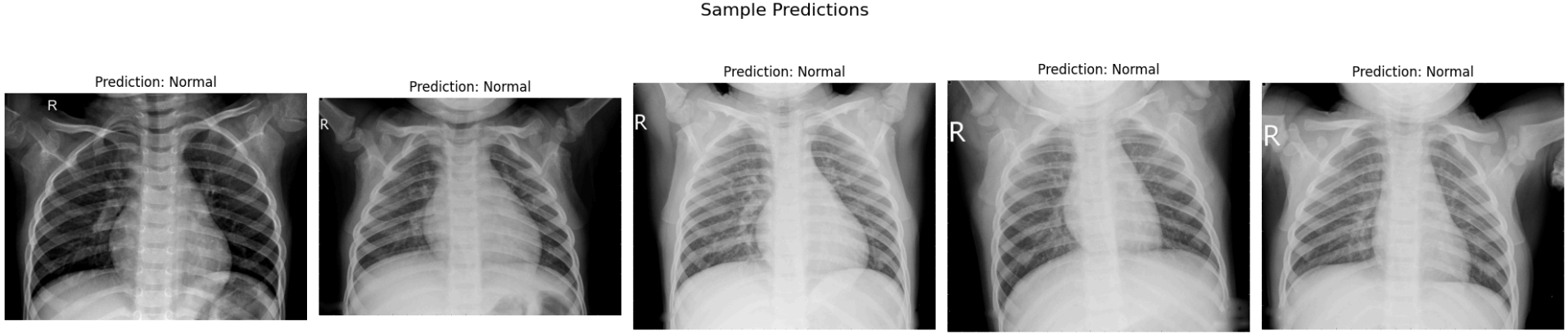
        axes[i].imshow(img, cmap='gray')
```

```
        prediction_for_sample = results_df[results_df['Image Path'] == image_file]['Prediction'].values[0]
        axes[i].set_title(f"Prediction: {prediction_for_sample}")
        axes[i].axis('off')

    except Exception as e:
        print(f"Error processing {image_file}: {e}")

plt.tight_layout()
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step
                                  Image Path Prediction
0  /content/drive/MyDrive/TA/Dataset/val/NORMAL/N...     Normal
1  /content/drive/MyDrive/TA/Dataset/val/NORMAL/N...     Normal
2  /content/drive/MyDrive/TA/Dataset/val/NORMAL/N...     Normal
3  /content/drive/MyDrive/TA/Dataset/val/NORMAL/N...     Normal
4  /content/drive/MyDrive/TA/Dataset/val/NORMAL/N...     Normal
5  /content/drive/MyDrive/TA/Dataset/val/NORMAL/N...     Normal
6  /content/drive/MyDrive/TA/Dataset/val/NORMAL/N...     Normal
7  /content/drive/MyDrive/TA/Dataset/val/NORMAL/N...     Normal
```



OTHER EXPERIMENT

```
import matplotlib.pyplot as plt
import cv2
```

```
import pandas as pd
import os
import random

test_dir = '/content/drive/MyDrive/TA/Dataset/val/PNEUMONIA'
image_files = [os.path.join(test_dir, f) for f in os.listdir(test_dir) if os.path.isfile(os.path.join(test_dir, f))]

image_paths = []
predictions = []

for image_file in image_files:
    try:
        new_image = load_img(image_file, target_size=(150, 150))
        new_image_array = img_to_array(new_image)
        new_image_array = np.clip(new_image_array - 30, 0, 255)
        new_image_array = new_image_array / 255.0
        new_image_glcm = extract_glcm_features([new_image_array])
        new_image_glcm_df = pd.DataFrame(new_image_glcm)
        new_image_scaled = scaler.transform(new_image_glcm_df)
        prediction = model.predict(new_image_scaled)
        predicted_class = (prediction > 0.5).astype(int)[0][0]

        image_paths.append(image_file)
        predictions.append('Pneumonia' if predicted_class == 1 else 'Normal')

    except Exception as e:
        print(f"Error processing {image_file}: {e}")

results_df = pd.DataFrame({'Image Path': image_paths, 'Prediction': predictions})
print(results_df)

sample_files = random.sample(image_files, 5)

fig, axes = plt.subplots(1, 5, figsize=(20, 5))
fig.suptitle('Sample Predictions', fontsize=16)

for i, image_file in enumerate(sample_files):
    try:
        img = cv2.imread(image_file, cv2.IMREAD_GRAYSCALE)

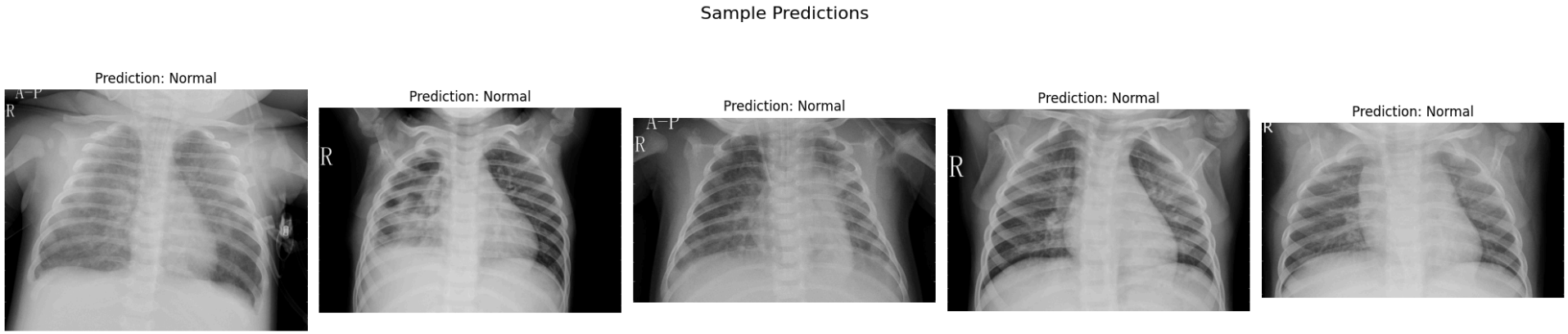
        axes[i].imshow(img, cmap='gray')
        prediction_for_sample = results_df[results_df['Image Path'] == image_file]['Prediction'].values[0]
        axes[i].set_title(f"Prediction: {prediction_for_sample}")
        axes[i].axis('off')

    except Exception as e:
```

```
        print(f"Error processing {image_file}: {e}")

plt.tight_layout()
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step
                                          Image Path Prediction
0  /content/drive/MyDrive/TA/Dataset/val/PNEUMONI...     Normal
1  /content/drive/MyDrive/TA/Dataset/val/PNEUMONI...     Normal
2  /content/drive/MyDrive/TA/Dataset/val/PNEUMONI...     Normal
3  /content/drive/MyDrive/TA/Dataset/val/PNEUMONI...     Normal
4  /content/drive/MyDrive/TA/Dataset/val/PNEUMONI...     Normal
5  /content/drive/MyDrive/TA/Dataset/val/PNEUMONI...     Normal
6  /content/drive/MyDrive/TA/Dataset/val/PNEUMONI...  Pneumonia
7  /content/drive/MyDrive/TA/Dataset/val/PNEUMONI...  Pneumonia
```



Tes SVM

```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Create and train the SVM model
svm_model = SVC(kernel='linear', random_state=42) # You can choose different kernels
svm_model.fit(X_train, y_train)
```

```
# Make predictions on the test set
y_pred_svm = svm_model.predict(X_test)

# Evaluate the SVM model
print("\n--- SVM Model Evaluation ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred_svm):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_svm))

# Confusion Matrix
conf_matrix_svm = confusion_matrix(y_test, y_pred_svm)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_svm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Pneumonia'], yticklabels=['Normal', 'Pneumonia'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix (SVM)')
plt.show()
```